

Code Review — cypressAllure

Repository: github.com/AZANIR/cypressAllure (Cypress 10 + TypeScript + Allure e2e framework) · Branch: `master` @ `bb313ea` · Reviewed: 2026-10-05 · Reviewer: Hryhorii Markevych

Request changes — the suite cannot run green today

4
BLOCKER

13
MAJOR

27
MINOR

Review summary — top-level comment

This is a small Cypress + TypeScript + Allure framework (2 page objects, 2 spec files, 6 tests). The Page Object Model is present and no locators leak into spec files — a good foundation. However, the suite cannot produce a meaningful green run today: the application under test (automationpractice.com) is gone and now serves a hosting placeholder page, one test asserts a typo'd string and can only fail, another test has no assertion and can only pass, and the CI workflow targets a retired runner image and retired action versions, so it can no longer start. On top of that, a generated Allure report (81 files including 7 .mp4 recordings) is committed into `docs/` on `master` by the CI itself, and the deploy step's secret is quoted into a literal string.

Recommended before merge

- Repoint the suite at a reachable target (self-hosted OpenCart/PrestaShop, another public demo, or a mock), or mark the repo as unmaintained.
- Fix the typo'd expected message and replace the `console.log "Sample Test"` with a real assertion (or delete it).
- Modernise the workflow (`ubuntu-latest`, `checkout@v4`, `setup-node@v4`, `upload-artifact@v4`, `npm ci`).
- Fix the secret interpolation **together with** moving the deploy target to `gh-pages` — fixing only the secret would create a self-triggering loop (see the deploy finding).
- Remove `docs/` from `master`, extend `.gitignore`, and unify the Allure results path used by the `report` script.
- Apply the POM improvements: module-level selector constants, a base page class, assertions kept in specs.

How the findings were verified

- Every file was read at commit `bb313ea`; line numbers refer to that commit.
- The repo's own committed Allure report (`docs/`, last CI run 2022-07-16) was used as run evidence: 6 tests, 5 passed, 1 failed. It shows the Allure writer *does* run in CI and records the exact failure message of the typo test.
- Git history: 12 "Deploying to master from master" commits, each directly after a source push and never followed by another deploy — i.e. deploys succeed and there is no re-trigger loop today.
- Live check on 2026-10-05: `http://automationpractice.com/index.php` returns an InMotion Hosting "HOSTNAME" placeholder page (HTTP 406 ModSecurity for non-browser clients).

Blocker findings (4)

BLOCKER config/config.config.ts:22

Test target

The suite's target site no longer serves the application under test

```
22 -   baseUrl: 'http://automationpractice.com/index.php',
```

Why: a live request today returns an InMotion Hosting "HOSTNAME" placeholder instead of the demo shop, so every `cy.visit('/')` lands on a page without `.login`, `#email`, etc. The committed report shows the site was already unstable in July 2022 (several runs broken with "508: Loop Detected"). No test can pass until the suite points at a reachable target — a self-hosted instance, a different public demo, or a mock server.

BLOCKER cypress/e2e/tests/login.test.ts:35

Test quality

Expected error text has a typo — this test can never pass

```
33   it('login with wrong email format credentials read data from fixture', function () {
34     loginPage.login(this.data.invalid_credentials.wrong_email_format.emailId, ...)
35 -   loginPage.validateLoginError('Invalid email addresssssss.')
35 +   loginPage.validateLoginError('Invalid email address.')
```

Why: the repo's own report records the failure: *"expected '' to have text 'Invalid email addresssssss.', but the text was 'Invalid email address.'"*. A permanently red test trains the team to ignore red CI runs. Verify expected strings against the app before committing.

BLOCKER cypress/e2e/tests/myAccount.test.ts:8-10

Test quality

`console.log` used instead of an assertion — the test always passes and checks nothing

```
8 -   it('Sample Test', () => {
9 -     console.log("This is a sample test")
10 -   })
8 +   it('shows the account page heading after login', function () {
9 +     loginPage.launchApplication()
10 +     loginPage.login(this.users.valid_credentials.emailId, this.users.valid_credentials.password)
11 +     myAccountPage.pageHeading.should('contain.text', 'My account')
12 +   })
```

Why: `console.log` never fails a test and does not surface in results — "Sample Test" is reported as *passed* in the committed report although it verifies nothing. It inflates the pass rate (5/6 looks healthier than 4/5). Give it a real `should` / `expect` or delete it.

The workflow can no longer run on GitHub-hosted runners

```
9 - runs-on: ubuntu-20.04
12 - node-version: [14.17.0]
15 - - uses: actions/checkout@v2
17 - uses: actions/setup-node@v2
21 - - run: npm i npm@latest
29 - uses: actions/upload-artifact@v1

9 + runs-on: ubuntu-latest
12 + node-version: [20.x]
15 + - uses: actions/checkout@v4
17 + uses: actions/setup-node@v4
21 + # removed - setup-node already ships a current npm
29 + uses: actions/upload-artifact@v4
```

Why: the `ubuntu-20.04` runner image was retired by GitHub in 2025, so jobs requesting it are never scheduled; `upload-artifact@v1` is retired as well and `checkout@v2` / `setup-node@v2` run on deprecated Node runtimes. Node 14 is end-of-life. `npm i npm@latest` additionally adds npm itself as a project dependency. Until this is updated, CI gives no signal at all.

Major findings (13)

A fully generated Allure report is committed to `master`

```
docs/
  data/test-cases/*.json      (30 files)
  data/attachments/*.mp4     (7 video recordings)
  data/attachments/*.png     (5 screenshots)
  history/ (5) widgets/ (14) export/ (3) plugins/ (4)
  app.js, styles.css, index.html, favicon.ico
+ git rm -r docs/
+ # add "docs/" to .gitignore; publish reports from CI to gh-pages instead
```

Why: generated output (test-case JSON, screenshots, videos) does not belong in the source branch — it bloats every clone and buries real changes: 12 of the repo's 56 commits are report deploys. (Checklist #5: never commit generated artifacts.)

The deploy secret is quoted into a string literal — the configured token is never used

```
38 - ACCESS_TOKEN: ${{ 'secrets.ACCESS_TOKEN' }}
38 + ACCESS_TOKEN: ${{ secrets.ACCESS_TOKEN }}
```

Why: quoting turns the expression into the literal text `secrets.ACCESS_TOKEN`. Deploys succeeded anyway (12 deploy commits), evidently because the push was authenticated by the `GITHUB_TOKEN` that `actions/checkout` persists in the local git config — so the step works by accident, not by configuration. **Do not apply this fix alone:** see the next finding.

MAJOR .github/workflows/cyAllureRun.yml:3-5, 34-41

CI

CI deploys the report back into `master`, the branch that triggers the workflow

```
4     push:
5       branches: [master]
...
39 -     BRANCH: master # The branch the action should deploy to.
41 -     TARGET_FOLDER: docs
39 +     BRANCH: gh-pages
41 +     # TARGET_FOLDER removed - publish at the gh-pages root
```

Why: today there is no loop — the history shows exactly one deploy commit per source push, because pushes made with `GITHUB_TOKEN` do not trigger new workflow runs. But as soon as the secret above is fixed and a personal access token is really used, each deploy commit to `master` will re-trigger this workflow and deploy again, indefinitely. Ship both fixes together and publish to a dedicated `gh-pages` branch (or `actions/deploy-pages`).

MAJOR package.json:10-11, config/config.config.ts:14

Config

The local `report` script reads a results folder that nothing writes

```
config/config.config.ts
14 -   allureResultsPath: '../allure-results',
14 +   allureResultsPath: 'allure-results',
package.json
10 - "report": "npx allure generate reports/ui/allure-results --clean -o reports/ui/allure-report &&
allure open reports/ui/allure-report",
10 + "report": "npx allure generate allure-results --clean -o allure-report && npx allure open allure-
report",
11   "allure:generate": "npx allure generate allure-results --clean",
```

Why: three different spellings of the results path exist. The CI path works — `allure:generate` reads `allure-results` and the deployed report is populated, so `'../allure-results'` evidently resolves to the project root. But `report`, which is the last step of `cy:e2e:run`, reads `reports/ui/allure-results`, which is never written — the local one-command run always opens an empty report. Use one plain path everywhere.

MAJOR package.json:12

Build

`clear` script is POSIX-only, so `cy:e2e:run` fails on Windows

```
12 -   "clean": "rm -r reports/** || true"
12 +   "clean": "rimraf reports allure-results allure-report"
```

Why: npm runs scripts through `cmd.exe` on Windows, where neither `rm` nor `true` exist, and `clear` is the first step of `cy:e2e:run`. Add `rimraf` as a devDependency so the run script works on every OS. It should also clean `allure-results`, otherwise results from previous runs are merged into the next report.

MAJOR cypress/fixtures/users.json:2-5

Security

Credentials committed to version control in plaintext

```
2 -   "valid_credentials": {
3 -     "emailId": "testautomation@cypressstest.com",
4 -     "password": "Test@1234"
```

Why: even for a throwaway demo account, credentials in a tracked file stay in git history forever and get copied into new projects as a pattern. Read them from `Cypress.env('USER_EMAIL')` / `Cypress.env('USER_PASSWORD')`, supplied by CI secrets or a git-ignored `cypress.env.json`; keep only non-secret negative-path data in the fixture.

MAJOR tsconfig.json:7, cypress/e2e/tests/login.test.ts:7-8, 18

Test quality

`strict: true` is enabled, but fixture data flows through the tests untyped

```
7 -     cy.fixture('users.json').then(function (data) {
8 -       this.data = data;
+ interface Users { valid_credentials: Credentials; invalid_credentials: Record<string, Credentials> }
+ cy.fixture<Users>('users.json').as('users')
```

Why: `this.data` is untyped Mocha context, so every access such as `this.data.invalid_credentials.wrong_email_format.password` bypasses the type checker — a renamed fixture key fails only at runtime. Typing the fixture makes `strict` actually protect the data the tests depend on most.

MAJOR .github/workflows/cyAllureRun.yml:22

CI

CI installs with `npm i` instead of `npm ci`

```
22 -       - run: npm i
22 +       - run: npm ci
```

Why: `npm i` can resolve different transitive versions than `package-lock.json` and may rewrite the lockfile in CI. `npm ci` installs exactly what is locked and fails fast when lockfile and `package.json` drift.

MAJOR cypress/e2e/pages/loginPage.ts:4-9, myAccountPage.ts:6-7

Page Objects

Selectors are embedded in class getters instead of module-level constants

```
4 -   get signinLink() { return cy.get('.login') }
5 -   get emailAddressTxt() { return cy.get('#email') }
6 -   get passwordTxt() { return cy.get('#passwd') }
7 -   get signinBtn() { return cy.get('#SubmitLogin') }
1 + const SIGNIN_LINK = '.login'
2 + const EMAIL_INPUT = '#email'
3 + const PASSWORD_INPUT = '#passwd'
4 + const SUBMIT_LOGIN_BTN = '#SubmitLogin'
... inside the class: cy.get(EMAIL_INPUT).type(emailId)
```

Why (stated fairly): these are lazy getters, so `cy.get()` resolves only when read — there is no stale-element problem here. The cost is that selector strings are hidden inside query calls and cannot be reused (e.g. by a base-page helper) or reviewed in one place. Module-level constants fix that at no cost. (Checklist #1.)

No base page / shared interaction layer

cypress/support/pages/basePage.ts (new)

```
1 + export class BasePage {
2 +   protected clickElement(selector: string) { cy.get(selector).click() }
3 +   protected fillField(selector: string, value: string) { cy.get(selector).clear().type(value) }
4 +   getElement(selector: string) { return cy.get(selector) }
5 + }
```

Why: both page objects call `cy.get()` directly. A two-level POM centralises raw Cypress calls, avoids repeating the same boilerplate in every new page and gives one place for cross-cutting concerns (logging, Allure steps, custom waits). (Checklist #3.)

Hardcoded credentials duplicate the fixture, and all negative data is static

```
12 - loginPage.login("testautomation@cypresstest.com", "Test@1234")
12 + loginPage.login(this.users.valid_credentials.emailId, this.users.valid_credentials.password)
+ // negative cases: faker.internet.email(), faker.internet.password()
```

Why: the next test reads the same values from `users.json`, so the credential lives in two places that can drift. The negative-path data (unknown email, wrong password) is identical on every run — the pesticide paradox. Generate non-critical values with `@faker-js/faker` (not present in `package.json`). (Checklist #4.)

Spec visits an unrelated absolute URL, imports an unused page object and keeps dead code

```
1 - import { loginPage } from "../pages/loginPage";
3 describe('My Account Functionality', () => {
4   beforeEach(() => {
5 -     cy.visit('https://google.com');
6 -     //loginPage.launchApplication()
```

Why: `loginPage` is referenced only by the commented-out line; visiting `google.com` in a "My Account" suite while a `baseUrl` is configured shows the file was left mid-edit. Together with the `console.log` test, this spec covers nothing about My Account.

MyAccountPage reaches into LoginPage's locator

```
17 - public validateSuccessfulLogout() {
18 -   loginPage.signinLink.should('be.visible')
19 - }
cypress/e2e/tests/login.test.ts
+ myAccountPage.logout()
+ loginPage.signinLink.should('be.visible')
```

Why: "logged out" is a statement about the login page's state, not the account page. Importing one page object into another couples them; the spec should call `logout()` and then assert on `loginPage`.

Minor findings (27)

MINOR config/config.config.ts:17-21, cypress/plugins/index.js

Config

Allure writer is loaded through a bare specifier from the legacy Cypress 9 plugins file

```
17 - // We've imported your old cypress plugins here.
18 - // You may want to clean this up later by importing these.
20 - return require('cypress/plugins/index.js')(on, config)
+ import allureWriter from '@shelex/cypress-allure-plugin/writer'
20 + allureWriter(on, config); return config
```

Why: 'cypress/plugins/index.js' reads like a path inside the `cypress` npm package. The committed report proves it currently resolves to the project's own file (via the tsconfig `baseUrl: "./"` mapping), but that is non-obvious and breaks if `baseUrl` changes. The config's own migration comment asks for this cleanup: inline the writer in `setupNodeEvents` and delete `cypress/plugins/`.

MINOR package.json:10

Build

Inconsistent `npx` usage inside the same script

```
10 "report": "npx allure generate ... && allure open reports/ui/allure-report",
```

Why: this works — npm puts `node_modules/.bin` on `PATH` for scripts and `allure-commandline` is a `devDependency` — but mixing `npx allure` and bare `allure` in one line suggests a global install is expected. Pick one style (checklist #7 recommends `npx`).

MINOR config/config.config.ts

Config

No retries configured against a shared public demo site

```
+ retries: { runMode: 2, openMode: 0 },
```

Why: the suite depends on a third-party site outside the team's control (the July 2022 report already shows "508: Loop Detected" runs). Run-mode retries absorb transient network failures; Allure still shows the retry history.

MINOR cypress/e2e/pages/

Structure

Page objects live inside the spec root

Why: `pages/` under `cypress/e2e/` only stays out of the run because `specPattern` is narrowed to `tests/**/*.test.ts`. Widening the pattern later would make Cypress try to execute page objects as specs. Conventional location: `cypress/support/pages/` or `cypress/pages/`.

MINOR package.json:8-9

Config

Cypress config lives at a non-default path

```
9 "cy:run": "cypress run --config-file config/config.config.ts --env allure=true",
```

Why: Cypress auto-discovers `cypress.config.ts` at the project root. With `config/config.config.ts`, every plain `npx cypress open`, IDE plugin or third-party CI action silently runs with defaults (no `baseUrl`, default `specPattern`). There is no benefit to the custom location here.

MINOR tsconfig.json:12-14

Build

include points one directory above the project root

```
13 -   "../node_modules/@shelex/cypress-allure-plugin/reporter",
14 -   "../node_modules/cypress"]
+   "types": ["node", "cypress", "@shelex/cypress-allure-plugin"]
```

Why: `../node_modules` escapes the repository, so these entries match nothing and the Allure plugin's `cy.allure()` typings are not loaded. Harmless today only because no spec calls `cy.allure()`.

MINOR cypress/e2e/pages/loginPage.ts:8

Page Objects

Error-box locator is coupled to visible text

```
8 -   get alertBox() { return cy.get('p:contains("error")') }
8 +   get alertBox() { return cy.get('.alert-danger') }
```

Why: any copy change ("error" → "problem", different casing) silently breaks this locator; the `.alert-danger` class used on the next line is the stable target.

MINOR cypress/e2e/pages/myAccountPage.ts:10

Test quality

Exact-text assertion is whitespace-sensitive

```
10 -   this.pageHeading.should('have.text', 'My account')
10 +   this.pageHeading.should('contain.text', 'My account')
```

Why: `have.text` requires an exact match including surrounding whitespace and nested markup; `contain.text` survives incidental template changes.

MINOR cypress/e2e/tests/myAccount.test.ts:6

Repo hygiene

Commented-out code left in place

```
6 -   //loginPage.launchApplication()
```

Why: dead code should be deleted, not commented out — git keeps the history. (Checklist #6.)

MINOR cypress/support/commands.js, cypress/support/e2e.js

Repo hygiene

Scaffold boilerplate never cleaned up; support files are JS in a TS project

Why: both files still carry the default Cypress template comments ("This example commands.js shows you how to..."); `commands.js` defines nothing. Trim them to what is used (in `e2e.js` that is the single plugin import) and convert to `.ts` for consistency.

MINOR .gitignore

Repo hygiene

Redundant entries, while the folder that was actually committed is not ignored

```
2  videos
3  reports
4  screenshots
6 - reports/**
+  docs/
+  cypress/screenshots
+  cypress/videos
```

Why: `reports/**` duplicates `reports`; meanwhile `docs/` — the 81 generated files on `master` — is not ignored.

MINOR cypress/e2e/tests/login.test.ts:5-10

Test quality

Fixture reloaded before every test, including the one that doesn't use it

```
7 -     cy.fixture('users.json').then(function (data) {
8 -         this.data = data;
9 -     })
+   beforeEach(() => cy.fixture('users.json').as('users'))
```

Why: a Cypress alias (`this.users`) is the idiomatic form and reads more clearly than assigning to Mocha context by hand.

MINOR cypress/e2e/pages/loginPage.ts:22-26, myAccountPage.ts:9-11

Page Objects

Assertions live inside page object methods instead of the spec

```
22  public validateLoginError(errorMessage: string) {
23      this.alertBox.should('be.visible')
24      this.alertMessage.should('have.text', errorMessage)
```

Why: page objects should expose elements and actions; keeping `should` / `expect` in specs makes the checked behaviour visible in the test and lets `alertMessage` be reused for other checks without new `validate*` methods. (Checklist #2.)

MINOR project-wide

Reporting

No `cy.allure()` usage anywhere

Why: the project is Allure-branded, yet no spec sets severity, epic/feature/story, owner or steps — the committed report is a flat list of six test names with no structure.

MINOR project-wide

Repo hygiene

No linter/formatter; inconsistent style

```
login.test.ts:8      this.data = data;           (semicolon)
login.test.ts:12     loginPage.login(...)       (none)
loginPage.ts:8       ...cy.get('p:contains("error")') } (missing space)
```

Why: no ESLint (`eslint-plugin-cypress`) or Prettier config exists. A formatter plus a `lint` script in CI keeps style from drifting.

MINOR README.md:21-29

Docs

Run instructions are vague and don't match the scripts

```
25 - Change "Username" and "Password"
```

Why: no file is named for where to change credentials, and the README does not mention `cy:e2e:run`, `report` or `allure:generate`, or where results end up.

MINOR cypress/e2e/tests/login.test.ts:11-22

Coverage

Two tests cover the identical happy path

```
11 it('login with valid credentials', function () {  
17 it('login with valid credentials read data from fixture', function () {
```

Why: same account, same assertions, same logout — only the data source differs. Once the hardcoded version reads the fixture, one of them is redundant.

MINOR cypress/e2e/tests/login.test.ts:13-15

Test quality

Each login test also asserts logout

```
13 myAccountPage.validateSuccessfulLogin()  
14 myAccountPage.logout()  
15 myAccountPage.validateSuccessfulLogout()
```

Why: when "login with valid credentials" fails, the name doesn't tell whether login or logout broke. A dedicated logout test keeps each failure message meaningful.

MINOR package.json:4-5, 22-31

Repo hygiene

Package metadata is template leftovers

```
4 - "description": "\"Automation Testing framework using cypress and typescript with allure-report\"",  
5 - "main": "index.js",
```

Why: the description contains escaped quotes, `main` names a file that does not exist, and the package is not marked `"private": true` although it is not meant to be published.

MINOR config/config.config.ts:5-10

Config

Timeouts cut below defaults without explanation; others repeat defaults

```
6 execTimeout: 5000,           (default 60000)  
7 taskTimeout: 5000,          (default 60000)  
8 pageLoadTimeout: 30000,     (default 60000)  
9 requestTimeout: 5000,       (= default)  
10 responseTimeout: 30000,     (= default)
```

Why: against a slow public demo, shortened page-load/exec timeouts produce flaky failures rather than faster feedback; the values equal to defaults are just noise. Keep only overrides that have a reason, and state it.

MINOR .github/workflows/cyAllureRun.yml

CI

Small CI setup gaps

Why: the workflow is named "...using Cypress Docker Image..." but uses no Docker image; the matrix has a single Node version, adding indirection without coverage; the Cypress binary (`~/.cache/Cypress`) is not cached; there is no `workflow_dispatch` trigger for manual re-runs; the deploy action `JamesIves/github-pages-deploy-action@3.1.0` is several majors old (current v4 uses `branch / folder` inputs).

MINOR cypress/e2e/pages/loginPage.ts:1, myAccountPage.ts:1

Repo hygiene

Redundant triple-slash reference directive

```
1 - /// <reference types="cypress" />
```

Why: `tsconfig.json` already declares `"types": ["node", "cypress"]` for the whole project.

MINOR cypress/e2e/tests/

Coverage

Thin functional coverage beyond credential negatives

Why: after removing the duplicate, the suite is one login/logout plus three near-identical "wrong credentials" variants (which could be one data-driven test). No coverage for empty-field submission, "Forgot your password?", session persistence or the My Account page content — worth a follow-up ticket.

MINOR README.md:1

Docs

Two headings run together on one line

```
1 - # cypress-e2e-allure# Cypress-e2e-Allure Automation-Framework with allure reports
1 + # Cypress E2E Allure – automation framework with Allure reports
```

Why: a missing newline merges two headings into one garbled title (the same stray `#` repeats on line 3).

MINOR README.md:17, 35

Docs

Screenshots hotlinked from a third-party image host

```
17 
35 
```

Why: if imgur purges the uploads, the README loses its only visuals. Commit the images (e.g. `.github/assets/`) or replace the structure screenshot with a text tree.

MINOR config/config.config.ts:22, loginPage.ts:12

Config

`baseUrl` contains a page path, so `cy.visit('/')` requests `index.php/`

```
22 - baseUrl: 'http://automationpractice.com/index.php',
22 + baseUrl: 'https://<app-origin>',
loginPage.ts
12 + cy.visit('/index.php')
```

Why: the report's failure messages show the resulting URL `http://automationpractice.com/index.php/` (trailing slash). `baseUrl` should be the origin; page paths belong in page objects. (Checklist #14.)

Toolchain is several major versions behind

```
36 "cypress": "^10.3.0",           (current: 16.x)
37 "typescript": "^4.7.4",         (current: 5.x)
34 "@types/node": "^18.0.5",
33 "@shelex/cypress-allure-plugin": "^2.28.0",
```

Why: Cypress 10 no longer receives fixes and doesn't support current browsers well. When upgrading, consider the vendor-maintained `allure-cypress` adapter instead of the community `@shelex` plugin.

What's done right

- A real Page Object Model exists for both pages instead of raw Cypress calls scattered through specs.
- No locators appear in spec files — every interaction goes through `loginPage / myAccountPage`.
- `baseUrl` is defined in config rather than repeated per test.
- `allure-commandline` is a devDependency, not a global install; `package-lock.json` is committed.
- Most login tests read data from `cypress/fixtures/users.json` rather than inlining it (one exception noted).
- TypeScript with `strict: true` is a good baseline, even though fixture typing undercuts it today.
- CI is set up to run the suite and publish an Allure report on every push — the intent is right; the implementation needs the fixes above.